Resumo: Prova 1 - Cache, Memória Virtual e Pipeline

Prefixo de Unidades

- 1 Byte (1B) = 8 bits (8b)

Nome	Símbolo	Base 2	Base 10
bit	b	2^0	10^0
quilobit	kb	2^{10}	10^3
megabit	Mb	2^{20}	10^6
gigabit	Gb	2^{30}	10^9
terabit	Tb	2^{40}	10^{12}

Memória

- **RAM**: Random-Access Memory.
- **ROM**: Read-Only Memory.
- **PROM**: Programmable ROM.
  - **EPROM**: apagável com radiação ultravioleta.
  - **EEPROM**: apagável por sinais elétricos.
- **Célula RAM Estática**: a memória estática é capaz de manter os bits de dados armazenados apenas enquanto a fonte de alimentação estiver conectada ao circuito. Uma célula *Static RAM* (SRAM) é equivalente ao *flip-flop*.
- **Célula RAM Dinâmica**: a *Dynamic RAM* (DRAM) é similar a SRAM. A diferença é o projeto das células. As células dinâmicas são **mais simples** e necessitam de **menos área no chip**. Isto permite que a DRAM seja construída com densidades de armazenamento maiores, reduzindo o custo do bit. A DRAM é muito **utilizada para memórias principais** dos computadores. A desvantagem da DRAM é que **as células são mais lentas**. Os tempos de leitura e escrita são maiores. Uma célula DRAM é construída a partir de *capacitores*, demandando *refresh* de memória para manter o os dados armazenados.

Hierarquia de Memória

- **Objetivo**: oferecer ilusão de máximo tamanho de memória, com mínimo custo.
- De **pequeno e rápido e maior custo** até **grande e lento e menor custo**, respectivamente:
  - **Chip do Processador**:
    - Banco de Registradores.
    - Memória Cache: L1, L2 e L3.
  - **Memória Primária**:
    - RAM, física, principal, real.
  - **Memória Secundária**:
    - Disco (HD), SSD.
- Cada nível contém cópia de parte da informação armazenada no nível superior seguinte.
- **Gerenciamento:**
  - Registradores $\longleftrightarrow$ Memória:
    - Pelo compilador.
  - Cache $\longleftrightarrow$ Memória Principal:
    - Pelo hardware.
  - Memória Principal $\longleftrightarrow$ Memória Secundária:
    - Pelo hardware e pelo SO (memória virtual).
    - Pelo programador (sistema de arquivos).
- **Princípio da Localidade**:
  - **Espacial**: se um dado é referenciado, seus vizinhos tendem a ser referenciados logo.
  - **Temporal**: um dado referenciado, tende a ser referenciado novamente.

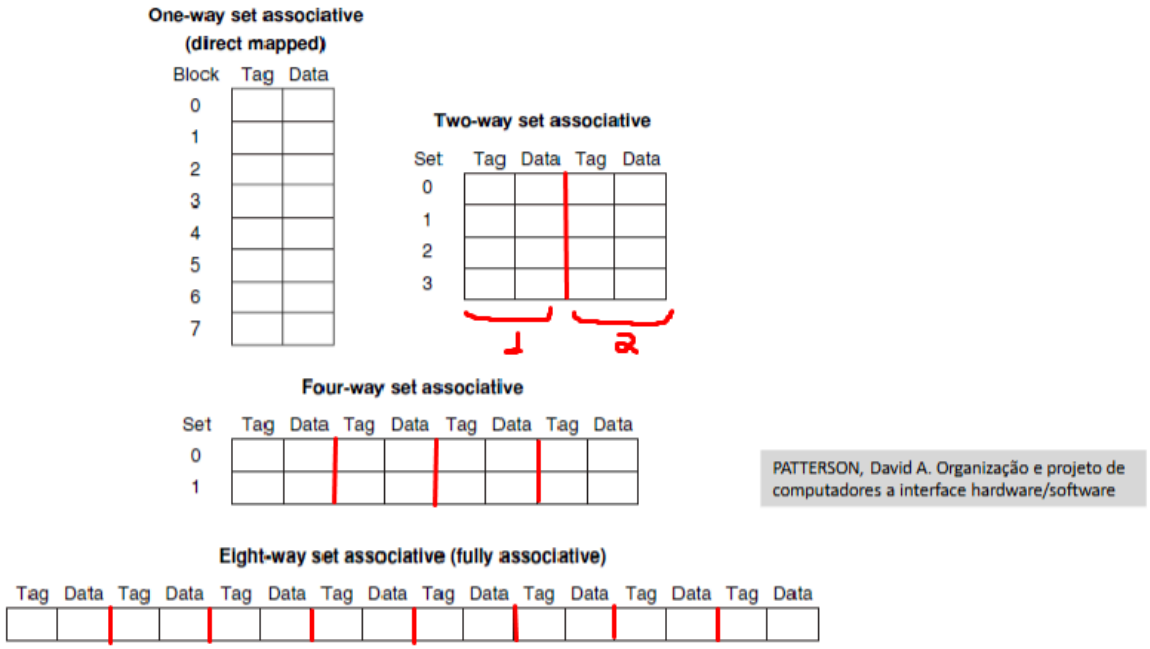
Memória Cache

- **Como Funciona**:
  - Processador gera endereço de memória e o envia à cache.
  - A Cache deve:
    - Verificar se tem cópia da posição de memória correspondente.
    - Se tem, encontrar a posição da cache onde está esta cópia.
    - Se não tem, trazer o conteúdo da memória principal e escolher posição da cache onde a cópia será armazenada.
- **Características**:
  - Pouco espaço de armazenamento.

    - Alto custo financeiro
    - Baixo tempo de acesso.
- **Conceitos**:
    - **Palavra**: conjunto de um ou mais Bytes.
    - **Bloco ou Linha**: conjunto de uma ou mais palavras (unidade da Cache).
    - **Bit de Válido**: indica se o dado ou bloco está válido.
    - **Tag ou rótulo**: parte do endereço de uma palavra na Memória Principal.
    - **Slot**: cada linha de uma Cache, que pode armazenar um ou mais blocos dependendo da organização da Cache.
    - **Comparador**: compara a tag de um endereço de uma palavra, com as tags dos endereços armazenados na Cache.

Estratégias de Organização (Mapeamento)

- **Mapeamento Direto**:
    - Cada dado só pode ir para **uma única linha**.
    - Simples, rápido, mas sujeito a **muitos conflitos**.
- **Mapeamento Conjunto-Associativo**:
    - A cache é dividida em **conjuntos**.
    - Cada conjunto tem N linhas e o dado pode ir em qualquer uma delas.
    - Reduz conflitos em relação ao direto.
- **Mapeamento Completamente Associativo**:
    - Qualquer dado pode ir para qualquer linha.
    - Quase sem conflitos, mas exige lógica de busca/substituição mais complexa.



PATTERSON, David A. Organização e projeto de computadores a interface hardware/software

Politica de Substituição

- **Objetivo**: substituir um bloco quando a Cache estiver sem espaço.
- **Mapeamento Direto**: o bloco que estiver no slot.
- **Set-Associativo e Completamente Associativo**:
  - **LRU (Least Recently Used)**: substituir o bloco menos recentemente utilizado.
    - Item (endereço de linha) vai para a frente da lista.
  - **LFU (Least Frequently Used)**: substituir o bloco menos frequentemente utilizado.
    - Contador incrementado quando bloco é acessado.
  - **FIFO (First In, First Out)**: substituir o primeiro bloco que entrou na cache.
  - **Aleatório (Random)**: escolher um bloco qualquer.

Política de Escrita

- **Objetivo**: manter a coerência Cache com a Memória Principal quando ocorrer escrita.
- **Write-Through**: a palavra é escrita tanto no bloco da Cache, quanto no bloco da Memória Principal.
- **Write-Back**: a palavra é escrita somente no bloco da cache e linha fica marcada como "suja". Quando este bloco for substituído, então a palavra será escrita na memória principal. Precisa de um bit de *dirty*.

Cache Hit/Miss

- **Cache Hit**: acerto na Cache, ou seja, o dado procurado já se encontra carregado na Cache.
  - **Hit Time**: tempo de acesso ao nível superior, que consiste de tempo de acesso + tempo para determinar hit/miss.
- **Cache Miss**: falta na Cache, ou seja, o dado procurado ainda tem que ser buscado na Memória Principal.
  - **Miss Penalty**: tempo gasto para substituir um bloco no nível superior + tempo para fornecer o bloco ao processador.
- **Métrica**:
  - **Taxa de acerto**: dado um número de acessos a cache, qual a porcentagem de cache hit.

$$\text{Taxa de Acerto} = \frac{\text{nº de cache hit}}{\text{nº de acessos}}$$

- **Tipos de Cache Miss**:
  - **Compulsório (Cold Start ou Chaveamento de Processos, Primeira Referência)**: primeiro acesso a uma linha.

    - Não se pode fazer algo a respeito.
  - **Conflito (ou Colisão)**: múltiplas linhas de memória acessando o mesmo conjunto da Cache Conjunto-Associativa ou mesma linha da Cache com Mapeamento Direto.
    - Solução: aumentar o tamanho da Cache ou aumentar a associatividade.
  - **Capacidade**: Cache não pode conter todas as linhas acessadas pelo programa.
    - Solução: aumentar o tamanho da Cache.
  - **Invalidação**: outro processo atualiza a memória.

Memória Virtual

- **Definição**:
  - É uma **técnica de gerenciamento de memória** que dá ao programa a ilusão de que ele tem **uma memória contínua e muito maior** do que a memória física (RAM) disponível.
  - Isso é feito armazenando parte dos dados na RAM e parte no **disco**, trazendo para a RAM apenas o que está sendo usado no momento.
- **Conceitos**:
  - **Endereço lógico (ou virtual)**: endereço gerado pela CPU ao executar um programa.
  - **Endereço físico**: endereço real na memória principal (RAM).
  - A conversão de **endereço lógico para físico** é feita automaticamente pelo hardware (MMU).

MMU (Memory Management Unit)

- A **MMU** é o componente do processador que faz a **tradução de endereços virtuais em endereços físicos**.
- Ela usa **estruturas de dados** chamadas **Tabelas de Páginas** (*Page Tables*) para saber onde cada página virtual está mapeada na RAM.
- Se a página não estiver na RAM $\rightarrow$ ocorre um **page fault** $\rightarrow$ o sistema operacional busca a página no disco e a coloca na RAM.

Paginação

- A memória virtual é dividida em **páginas** (virtual) e a memória física em **quadros** (frames).
- Exemplo: páginas de 4 kB $\rightarrow$ a memória RAM é dividida em quadros de 4 KB.
- Quando o programa pede um endereço virtual, a MMU procura na tabela de páginas:
  1. Se a página está na RAM $\rightarrow$ traduz e acessa diretamente (acerto).

2. Se a página não está → **page fault** → busca no disco → traz a página para um quadro livre (ou substitui uma página existente).

Gerência de Memória

- O **Sistema Operacional** é responsável por manter as **tabelas de páginas** de cada processo.
- Cada processo tem a impressão de que possui toda a memória, mas na prática apenas parte dela está realmente carregada na RAM.
- Para decidir **qual página remover** quando a RAM enche, usam-se algoritmos de substituição:
  - **FIFO (First In, First Out)** → substitui a mais antiga.
  - **LRU (Least Recently Used)** → substitui a menos recentemente usada.
  - **LFU (Least Frequently Used)** → substitui a menos acessada.
  - **Ótimo (OPT)** → substitui a página que não será usada por mais tempo (só usado teoricamente).

TLB (Translation Look-Aside Buffer)

- Traduzir um endereço virtual consultando a tabela de páginas na RAM é **caro**.
- Para acelerar isso, existe a **TLB**, que é uma **pequena cache dentro da MMU**.
- Ela guarda traduções recentes de endereços virtuais para físicos.
- Funcionamento:
  - **TLB hit**: a tradução está na TLB → acesso muito rápido.
  - **TLB miss**: precisa consultar a tabela de páginas na memória → mais lento.
- Assim como a cache, a TLB também usa **políticas de substituição** (FIFO, LRU, etc.).

MMTT e DMTT

- **MMTT (Main Memory Translation Table)**: tabela de tradução de páginas que está na memória principal (RAM). É consultada pela MMU em caso de **TLB miss**.
- **DMTT (Disk Memory Translation Table)**: tabela de tradução que referencia páginas que estão no disco (swap). É usada em caso de **page fault**, para localizar a página no disco e trazê-la para a RAM.

Fluxo Simplificado

1. CPU gera um endereço virtual.
2. **TLB** verifica se já tem a tradução:
   - **TLB hit** → endereço físico pronto.

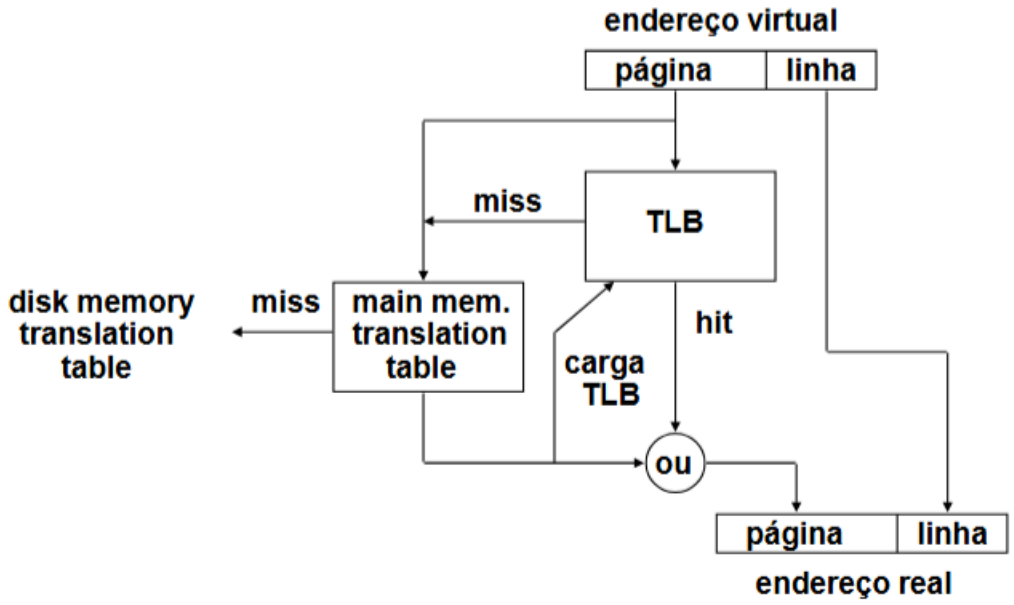
- **TLB miss** → consulta a **MMTT (tabela em RAM)**.

3. Se a página está na MMTT pega quadro físico na RAM.
4. Se não está → **page fault** → consulta a **DMTT** para descobrir onde a página está no disco → carrega para RAM → atualiza MMTT e TLB.

Fluxo Detalhado

1. **CPU** gera um **endereço virtual**.
2. **TLB** verifica se já existe a tradução:
   - **TLB Hit** → endereço físico obtido rapidamente → acesso direto à **RAM**.
   - **TLB Miss** → consultar a **MMTT** (tabela de páginas em RAM).
3. **Consulta à MMTT**:
   - **Entrada encontrada** → obtém quadro físico na **RAM** → atualização opcional da TLB → acesso concluído.
   - **Entrada não encontrada** → ocorre **Page Fault**.
4. **Page Fault**:
   - O SO consulta a **DMTT** para localizar a página no **disco**.
   - Carrega a página para um quadro livre na **RAM** (ou substitui outra página).
   - Atualiza a **MMTT** e a **TLB** com a nova tradução.
5. **Execução continua** com o endereço virtual traduzido para físico.

Fluxo em Imagem



Pipeline

- **Pipeline** é uma técnica de paralelismo que divide a execução de uma tarefa em várias etapas, permitindo que diferentes instruções sejam processadas simultaneamente, cada uma em um estágio diferente.

Pipelines Aritméticos

- Aplicados em **operações aritméticas complexas** (ex.: multiplicação, divisão).
- Uma operação é dividida em estágios menores $\rightarrow$ diferentes operandos podem entrar no pipeline antes que a operação anterior termine.
- **Ganho**: maior **throughput** (mais operações por unidade de tempo).

Pipelines de Instruções

- Usados em **processadores** para executar instruções de programas.
- **Exemplo clássico**: pipeline de 5 estágios do MIPS:
  1. IF — Instruction Fetch (busca da instrução)
  2. ID — Instruction Decode (decodificação e leitura de registradores)
  3. EX — Execute (execução ou cálculo de endereço)
  4. MEM — Memory (acesso à memória)
  5. WB — Write Back (escrita do resultado)

Desempenho

- O pipeline melhora o **throughput** (número de instruções por unidade de tempo), mas não reduz o tempo de execução de cada instrução individual.
- Medida importante:

$$\text{speedup} = \frac{\text{tempo sem pipeline}}{\text{tempo com pipeline}}$$

Conflitos de Memória

- Problema: quando duas instruções precisam acessar memória ao mesmo tempo, isto é, acessos simultâneos à memória por 2 ou mais estágios.
- Soluções: **cache separada** para instruções e dados.

Dependências em Desvios

- Problema: instruções de **desvio condicional (branch)** podem invalidar instruções já em execução no pipeline.

- Soluções:
  1. Executar os dois caminhos do desvio:

```
- Buffers paralelos de instruções.
```

  2. Prever o sentido do desvio:
     - **Predição Estática**:
       - É uma decisão tomada pelo compilador antes da execução do programa.
       - **Sempre tomada (predict-taken):** Assume que o desvio será sempre realizado.
       - **Nunca tomada (predict-not-taken):** Assume que o desvio nunca será realizado.
       - **Baseada no código:** A predição é definida com base em testes do código, como a probabilidade de desvios em loops.
       - Mais simples e não requer hardware adicional, porém menos preciso.
     - **Predição Dinâmica**:
       - O hardware usa informações de desvios anteriores para prever se um desvio será tomado ou não.
       - Utiliza um buffer de previsão de desvio (BHT - Branch History Table) que armazena o histórico de desvios de instruções.
       - Mais preciso, porém mais complexo e requer hardware dedicado.
  3. Eliminar o problema:
     - **Delayed branch**:
       - Desvio não ocorre imediatamente, e sim apenas após uma ou mais instruções seguintes.

Dependências Verdadeiras (RAW – Read After Write)

- Quando uma instrução precisa de um valor produzido por outra anterior que ainda não terminou.
- Exemplo:

```
I1: R1 <- R2 + R3
I2: R4 <- R1 + R5    (depende do resultado de I1)
```

- Se $\mathbf{I2}$ for executada antes de $\mathbf{I1}$ terminar $\rightarrow$ erro.

Dependências Falsas

- **WAR (Write After Read)** e **WAW (Write After Write)**.

- Acontecem quando o mesmo registrador é usado em instruções diferentes, mas sem relação lógica.
- Exemplo:

```
I1: R1 <- R2 + R3
I2: R1 <- R4 + R5    (WAW - sobrescreve antes do uso real)
```

Pipeline Interlock

- Método para manter sequência correta de leituras e escritas em registradores.
- Mecanismo de controle do hardware que **detecta dependências** e insere automaticamente *stalls* (bolhas) no pipeline até que a instrução anterior termine.

Forwarding (Adiantamento de Dados)

- Técnica que permite usar o resultado **assim que ele estiver disponível**, antes de ser escrito no registrador.
- O dado é “adiantado” diretamente do estágio de execução de uma instrução para o estágio que precisa dele.
- Reduz a necessidade de *stalls*.

FIM